

Experiment 'p_6_10_same_set_seq' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_10_same_set_seq

Natural language statement: Write a method that checks whether two arrays have the same elements in some order, ignoring duplicates.

Method signature: p_6_10_same_set_seq(first: seq<int>, second: seq<int>) returns (are_same_set: bool)

Ensures

- are_same_set == (array_to_set(first) == array_to_set(second))

Functional Code Given

```
function array_to_set(arr: seq<int>): set<int>
{
  if |arr| == 0 then {}
  else {arr[0]} + array_to_set(arr[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that checks whether two arrays have the same elements in some order,
→ ignoring duplicates.

The signature should be:

`p_6_10_same_set_seq(first: seq<int>, second: seq<int>) returns (are_same_set: bool)`

The method should respect the following contract:

`ensures are_same_set == (array_to_set(first) == array_to_set(second))`

The contract uses the following dafny code:

```
function array_to_set(arr: seq<int>): set<int>
{
  if |arr| == 0 then {}
  else {arr[0]} + array_to_set(arr[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn2XZD6yKoGHObWV9n6xzlGM8dcpy

Model: gpt-5.5-2026-04-23

Created at: 1780580209

Finish reason: stop

Usage: 430 tokens in, and 1590 tokens out

```
{
  are_same_set := (forall x: int :: x in first <==> x in second);
}
```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_6_10_same_set_seq/program.dfy(10,19): Error: quantifiers in non-ghost
  ↳ contexts must be compilable, but Dafny's heuristics can't figure out how to
  ↳ produce or compile a bounded set of values for 'x'
```

```
10 | are_same_set := (forall x: int :: x in first <==> x in second);
    | ~~~~~
```

1 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
↳ Dafny body, including the curly braces that surround it. Do not show the signature
↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn2Ybr2HU2ddjP7uSSsf1UNqCBgb3

Model: gpt-5.5-2026-04-23

Created at: 1780580273

Finish reason: stop

Usage: 606 tokens in, and 8571 tokens out

```
{
  var fSub := forall i: int | 0 <= i < |first| :: first[i] in second;
  var sSub := forall j: int | 0 <= j < |second| :: second[j] in first;
  are_same_set := fSub && sSub;

  ghost var sf: set<int> := {};
  var i := |first|;
  while 0 < i
    invariant 0 <= i <= |first|
    invariant sf == array_to_set(first[i..])
    invariant forall x: int :: x in sf <==> x in first[i..]
  {
    ghost var oldSf := sf;
    ghost var oldI := i;
    i := i - 1;
    assert oldI == i + 1;
    assert oldSf == array_to_set(first[i+1..]);
    assert first[i..] == [first[i]] + first[i+1..];
    assert |first[i..]| > 0;
    assert first[i..][0] == first[i];
    assert first[i..][1..] == first[i+1..];
    sf := {first[i]} + oldSf;
    assert array_to_set(first[i..]) == {first[i]} + array_to_set(first[i+1..]);
    assert sf == array_to_set(first[i..]);
    assert forall x: int :: x in sf <==> x in first[i..];
  }
  assert first[0..] == first;
  assert sf == array_to_set(first);
  assert forall x: int :: x in sf <==> x in first;
```

```

ghost var ss: set<int> := {};
var j := |second|;
while 0 < j
  invariant 0 <= j <= |second|
  invariant ss == array_to_set(second[j..])
  invariant forall x: int :: x in ss <==> x in second[j..]
{
  ghost var oldSs := ss;
  ghost var oldJ := j;
  j := j - 1;
  assert oldJ == j + 1;
  assert oldSs == array_to_set(second[j+1..]);
  assert second[j..] == [second[j]] + second[j+1..];
  assert |second[j..]| > 0;
  assert second[j..][0] == second[j];
  assert second[j..][1..] == second[j+1..];
  ss := {second[j]} + oldSs;
  assert array_to_set(second[j..]) == {second[j]} + array_to_set(second[j+1..]);
  assert ss == array_to_set(second[j..]);
  assert forall x: int :: x in ss <==> x in second[j..];
}
assert second[0..] == second;
assert ss == array_to_set(second);
assert forall x: int :: x in ss <==> x in second;

if fSub && sSub {
  forall x: int | x in sf
    ensures x in ss
  {
    assert x in first;
    var k :| 0 <= k < |first| && first[k] == x;
    assert first[k] in second;
    assert x in second;
    assert x in ss;
  }
  forall x: int | x in ss
    ensures x in sf
  {
    assert x in second;
    var k :| 0 <= k < |second| && second[k] == x;
    assert second[k] in first;
    assert x in first;
    assert x in sf;
  }
  assert sf <= ss;
  assert ss <= sf;
  assert sf == ss;
} else {
  if !fSub {
    var k :| 0 <= k < |first| && !(first[k] in second);

```

```

    assert first[k] in first;
    assert first[k] in sf;
    assert !(first[k] in ss);
    assert sf != ss;
  } else {
    assert !sSub;
    var k :| 0 <= k < |second| && !(second[k] in first);
    assert second[k] in second;
    assert second[k] in ss;
    assert !(second[k] in sf);
    assert sf != ss;
  }
}

assert are_same_set == (sf == ss);
assert are_same_set == (array_to_set(first) == array_to_set(second));
}

```

Final Program

```

function array_to_set(arr: seq<int>): set<int>
{
  if |arr| == 0 then {}
  else {arr[0]} + array_to_set(arr[1..])
}

method p_6_10_same_set_seq(first: seq<int>, second: seq<int>) returns (are_same_set: bool
  ↪ )
  ensures are_same_set == (array_to_set(first) == array_to_set(second))
{
  var fSub := forall i: int | 0 <= i < |first| :: first[i] in second;
  var sSub := forall j: int | 0 <= j < |second| :: second[j] in first;
  are_same_set := fSub && sSub;

  ghost var sf: set<int> := {};
  var i := |first|;
  while 0 < i
  invariant 0 <= i <= |first|
  invariant sf == array_to_set(first[i..])
  invariant forall x: int :: x in sf <==> x in first[i..]
  {
    ghost var oldSf := sf;
    ghost var oldI := i;
    i := i - 1;
    assert oldI == i + 1;
    assert oldSf == array_to_set(first[i+1..]);
    assert first[i..] == [first[i]] + first[i+1..];
    assert |first[i..]| > 0;
    assert first[i..][0] == first[i];
    assert first[i..][1..] == first[i+1..];
  }
}

```

```

    sf := {first[i]} + oldSf;
    assert array_to_set(first[i..]) == {first[i]} + array_to_set(first[i+1..]);
    assert sf == array_to_set(first[i..]);
    assert forall x: int :: x in sf <==> x in first[i..];
}
assert first[0..] == first;
assert sf == array_to_set(first);
assert forall x: int :: x in sf <==> x in first;

ghost var ss: set<int> := {};
var j := |second|;
while 0 < j
    invariant 0 <= j <= |second|
    invariant ss == array_to_set(second[j..])
    invariant forall x: int :: x in ss <==> x in second[j..]
{
    ghost var oldSs := ss;
    ghost var oldJ := j;
    j := j - 1;
    assert oldJ == j + 1;
    assert oldSs == array_to_set(second[j+1..]);
    assert second[j..] == [second[j]] + second[j+1..];
    assert |second[j..]| > 0;
    assert second[j..][0] == second[j];
    assert second[j..][1..] == second[j+1..];
    ss := {second[j]} + oldSs;
    assert array_to_set(second[j..]) == {second[j]} + array_to_set(second[j+1..]);
    assert ss == array_to_set(second[j..]);
    assert forall x: int :: x in ss <==> x in second[j..];
}
assert second[0..] == second;
assert ss == array_to_set(second);
assert forall x: int :: x in ss <==> x in second;

if fSub && sSub {
    forall x: int | x in sf
        ensures x in ss
    {
        assert x in first;
        var k :| 0 <= k < |first| && first[k] == x;
        assert first[k] in second;
        assert x in second;
        assert x in ss;
    }
    forall x: int | x in ss
        ensures x in sf
    {
        assert x in second;
        var k :| 0 <= k < |second| && second[k] == x;
        assert second[k] in first;
        assert x in first;
    }
}

```

```

    assert x in sf;
  }
  assert sf <= ss;
  assert ss <= sf;
  assert sf == ss;
} else {
  if !fSub {
    var k :| 0 <= k < |first| && !(first[k] in second);
    assert first[k] in first;
    assert first[k] in sf;
    assert !(first[k] in ss);
    assert sf != ss;
  } else {
    assert !sSub;
    var k :| 0 <= k < |second| && !(second[k] in first);
    assert second[k] in second;
    assert second[k] in ss;
    assert !(second[k] in sf);
    assert sf != ss;
  }
}

assert are_same_set == (sf == ss);
assert are_same_set == (array_to_set(first) == array_to_set(second));
}

```

Total Token Usage

Input tokens: 1036

Output tokens: 10161

Reasoning tokens: 8967

Sum of 'total tokens': 11197

Experiment Timings

Overall Experiment started at 1780580205125, ended at 1780580501834, lasting 296709ms (296.71 seconds)

Iteration #1 started at 1780580205130, ended at 1780580272945, lasting 67815ms (67.82 seconds)

Iteration #2 started at 1780580272952, ended at 1780580501828, lasting 228876ms (228.88 seconds)